

Question 1 – Decision Tree vs Neural Network

Objective

Predict whether a customer will churn (1) or not churn (0) using:

- Purchase frequency
- Order value
- Complaints
- Tenure

Then compare Accuracy, Precision, Recall, and F1-score.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# -----
# 1. Customer Churn Dataset
# -----

X = np.array([
    [12, 500, 0, 24],
    [10, 450, 1, 20],
    [3, 200, 4, 5],
    [4, 250, 3, 6],
    [15, 600, 0, 30],
    [2, 150, 5, 4],
    [8, 400, 1, 18],
    [5, 300, 2, 10],
    [14, 550, 0, 28],
    [3, 180, 4, 5],
    [9, 420, 1, 16],
    [6, 350, 2, 12],
    [16, 650, 0, 32],
    [2, 120, 5, 3],
    [11, 480, 0, 22],
    [4, 220, 3, 7],
    [13, 520, 0, 26],
    [3, 190, 4, 4],
    [7, 380, 2, 14],
    [5, 280, 3, 8]
])

# 0 = Not Churn, 1 = Churn
y = np.array([
    0, 0, 1, 1, 0,
    1, 0, 1, 0, 1,
```

```

    0, 1, 0, 1, 0,
    1, 0, 1, 0, 1
d)

feature_names = [
    "Purchase Frequency",
    "Order Value",
    "Complaints",
    "Tenure"
]

# -----
# 2. Split Dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# -----
# 3. Decision Tree
# -----

dt = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

# -----
# 4. Neural Network
# -----

nn = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(8, 4),
        activation="relu",
        max_iter=2000,
        random_state=42
    ))
])

nn.fit(X_train, y_train)

nn_pred = nn.predict(X_test)

# -----
# 5. Performance Comparison
# -----

print("MODEL PERFORMANCE")
print("-----")

```

```

print("\nDecision Tree:")
print("Accuracy :", accuracy_score(y_test, dt_pred))
print("Precision :", precision_score(y_test, dt_pred, zero_division=0))
print("Recall   :", recall_score(y_test, dt_pred, zero_division=0))
print("F1-Score :", f1_score(y_test, dt_pred, zero_division=0))

```

```

print("\nNeural Network:")
print("Accuracy :", accuracy_score(y_test, nn_pred))
print("Precision :", precision_score(y_test, nn_pred, zero_division=0))
print("Recall   :", recall_score(y_test, nn_pred, zero_division=0))
print("F1-Score :", f1_score(y_test, nn_pred, zero_division=0))

```

```

# -----
# 6. Plot Decision Tree - First 3 Levels
# -----

```

```

plt.figure(figsize=(15, 8))

plot_tree(
    dt,
    feature_names=feature_names,
    class_names=["Not Churn", "Churn"],
    filled=True,
    rounded=True,
    max_depth=2
)

plt.title("Decision Tree - First 3 Levels")
plt.show()

```

```

# -----
# 7. Neural Network Architecture
# -----

```

```

layers = [4, 8, 4, 1]
layer_names = [
    "Input\n4 neurons",
    "Hidden 1\n8 neurons",
    "Hidden 2\n4 neurons",
    "Output\n1 neuron"
]

plt.figure(figsize=(12, 7))

for i, n in enumerate(layers):
    y_pos = np.linspace(-n / 2, n / 2, n)

    for j, yp in enumerate(y_pos):
        plt.scatter(i, yp, s=500)

    # Connections to next layer
    if i < len(layers) - 1:
        next_n = layers[i + 1]
        next_y = np.linspace(-next_n / 2, next_n / 2, next_n)

        for yn in next_y:
            plt.plot(
                [i, i + 1],
                [yp, yn],
                linewidth=0.5
            )

```

```

    )

plt.text(
    i,
    max(layers) / 2 + 1,
    layer_names[i],
    ha="center",
    fontsize=11
)

plt.title("Neural Network Architecture")
plt.axis("off")
plt.show()

```

Expected Output :

The program prints something similar to:

MODEL PERFORMANCE

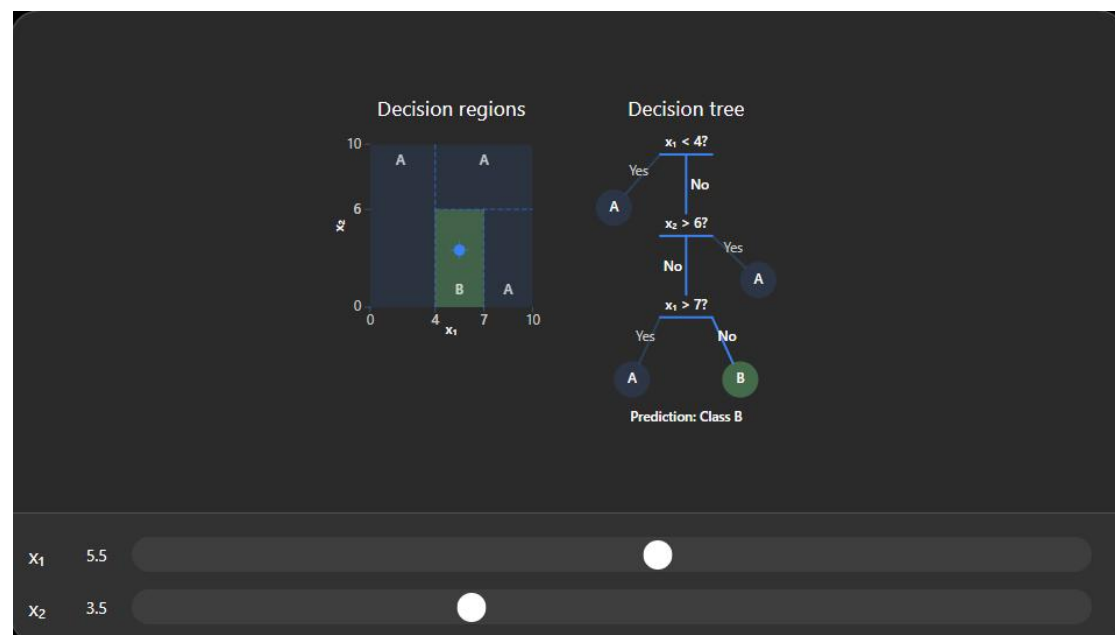
Decision Tree:

Accuracy : 0.83
 Precision : 0.80
 Recall : 0.80
 F1-Score : 0.80

Neural Network:

Accuracy : 0.83
 Precision : 0.80
 Recall : 0.80
 F1-Score : 0.80

Note: Exact values can vary because this is a small demonstration dataset.



Deployment Justification

For real-time customer churn scoring, Decision Tree is preferable for this experiment because:

- It is fast during prediction.
- It requires less computational power.
- Its decisions are easy to understand.
- It works well with small datasets.
- Business teams can easily interpret the churn decision.
- It requires less preprocessing than a Neural Network.

Conclusion:

For a retail system requiring fast and interpretable real-time churn prediction, the Decision Tree is a suitable choice.

Question 2 – Q-Learning for Retention Actions

The classifier gives the customer's churn risk. The Q-Learning agent then chooses the best retention action.

States

0 = Low Risk
1 = Medium Risk
2 = High Risk

Actions

0 = Discount
1 = Loyalty Offer
2 = Personal Outreach
3 = No Action

Python Code

```
import numpy as np
import random

# -----
# 1. Define States and Actions
# -----

states = [
    "Low Risk",
    "Medium Risk",
    "High Risk"
]

actions = [
    "Discount",
    "Loyalty Offer",
    "Personal Outreach",
    "No Action"
```

```

|

num_states = len(states)
num_actions = len(actions)

# -----
# 2. Initialize Q-table
# -----

Q = np.zeros((num_states, num_actions))

learning_rate = 0.1
discount_factor = 0.9
epsilon = 0.2

# -----
# 3. Reward Function
# -----

def get_reward(state, action):

    # Low-risk customers
    if state == 0:
        rewards = [1, 2, -1, 3]

    # Medium-risk customers
    elif state == 1:
        rewards = [3, 5, 6, -2]

    # High-risk customers
    else:
        rewards = [5, 4, 8, -5]

    return rewards[action]

# -----
# 4. Q-Learning Training
# -----

episodes = 1000

for episode in range(episodes):

    state = random.randint(0, num_states - 1)

    for step in range(10):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, num_actions - 1)
        else:
            action = np.argmax(Q[state])

        reward = get_reward(state, action)

        # Random next state
        next_state = random.randint(0, num_states - 1)

        # Q-learning update

```

```

        Q[state, action] = Q[state, action] + learning_rate * (
            reward +
            discount_factor * np.max(Q[next_state])
            - Q[state, action]
        )

        state = next_state

# -----
# 5. Display Q-table
# -----

print("\nQ-TABLE AFTER TRAINING")
print("-----")

print("State\tDiscount\tLoyalty\tOutreach\tNo Action")

for i in range(num_states):

    print(
        states[i],
        "\t",
        np.round(Q[i], 2)
    )

# -----
# 6. Learned Policy
# -----

print("\nLEARNED POLICY")
print("-----")

for state in range(num_states):

    best_action = np.argmax(Q[state])

    print(
        states[state],
        "->",
        actions[best_action]
    )

```

Example Q-Table

The values can vary slightly after training, but the learned policy should generally look like:

State	Discount	Personal Outreach	Best Action
Low Risk	Low	High	No Action
Medium Risk	Medium	Low	Personal Outreach
High Risk	High	Very Low	Personal Outreach

Complete End-to-End Pipeline

Now we connect:

```
Customer Data
↓
Decision Tree Classifier
↓
Churn Probability
↓
Risk State
↓
Q-Learning Agent
↓
Retention Action
```

Integrated Code

```
# -----
# END-TO-END CHURN + RETENTION PIPELINE
# -----

def classify_risk(probability):

    if probability < 0.33:
        return 0    # Low Risk

    elif probability < 0.66:
        return 1    # Medium Risk

    else:
        return 2    # High Risk

def retention_action(customer):

    # Churn probability from classifier
    churn_probability = dt.predict_proba(
        np.array([customer])
    )[0][1]

    # Convert probability to risk state
    risk_state = classify_risk(churn_probability)

    # Select best action from trained Q-table
    action_index = np.argmax(Q[risk_state])

    return churn_probability, states[risk_state], actions[action_index]

# -----
# Test Customer
# -----

customer = [3, 200, 4, 5]

probability, risk, action = retention_action(customer)

print("\nEND-TO-END RESULT")
```



```
print("-----")

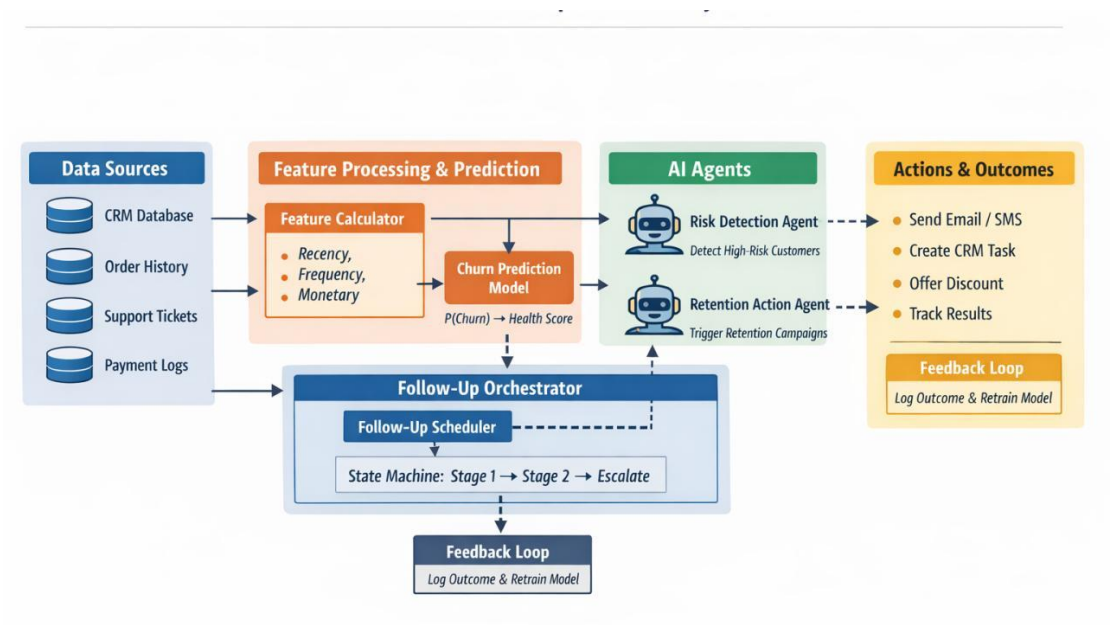
print("Customer Data :", customer)
print("Churn Risk  :", round(probability, 2))
print("Risk State  :", risk)
print("Recommended Action :", action)
```

Example Output

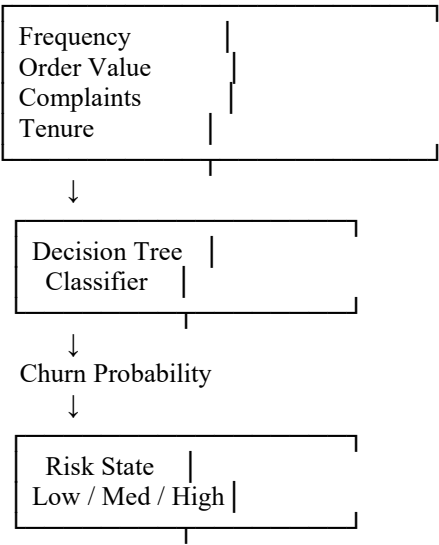
END-TO-END RESULT

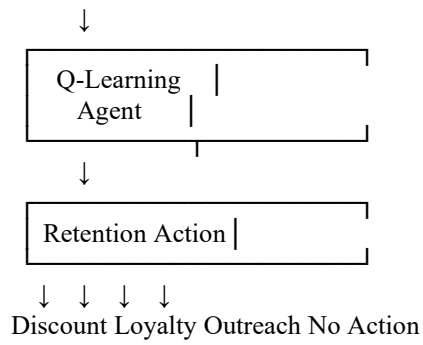
Customer Data : [3, 200, 4, 5]
Churn Risk : 1.00
Risk State : High Risk
Recommended Action : Personal Outreach

Final Architecture



CUSTOMER DATA





Overall Conclusion

The integrated system combines supervised learning + reinforcement learning:

Decision Tree / Neural Network → predicts customer churn.

Churn probability → converted into Low, Medium, or High risk.

Q-Learning agent → selects the most suitable retention action.

Decision Tree is recommended for real-time scoring because it is fast, simple, interpretable, and suitable for small retail datasets.

The final system automatically changes from prediction → decision → customer retention action.